# 自作言語で作った自作 CPU の上で自作 Ruby が動かない

全部自分のコードのとき、どこを疑うか

284km ・ YAPC::Tokyo 2026

# 最初に、ひとつ選んでください

このあと出てくる 3 つの容疑者のうち、犯人はどれか

最後に答え合わせをします

# 3 つの層、全部自分のコード

1. 自作 Ruby 処理系（4 万行超）
2. 自作コンパイラが出した RISC-V の機械語
3. 自作 CPU エミュレータ

Ruby 処理系も CPU エミュレータも、自作言語 Mere で書いた

CPU を 32 ビットから 64 ビットに広げた

# puts 1 + 1

が通らない

# 手を入れたのは、出す側と動かす側の両方

- 機械語を出すコンパイラと、それを動かす CPU エミュレータ
- 書き換えは 149 箇所
- 1 つのコミットではない

# 定石どおり、境界はテストしてあった

| 層 | 何と比べたか | 結果 |
| --- | --- | --- |
| 1. Ruby 処理系 | 本家 ruby 3.2.2、162 本 | バイト単位で一致 |
| 2. 機械語 | 同じプログラムを C に変換して実行 | 64 / 32 ビットとも不一致 0 |
| 3. CPU エミュレータ | 同じ機械語を QEMU で実行 | 7 / 7 一致 |

2026 年 9 月初め。全部、緑

境界のテストが言えること

# 試したプログラムでは、一致する

「その層が正しい」とは、言っていない

# 容疑者は 3 人とも残る

1. 自作 Ruby 処理系
2. 自作コンパイラが出した機械語
3. 自作 CPU エミュレータ

どれだと思いますか

# 他人の層なら、材料が外から来る

・issue、既知のバグ、changelog

・「みんな使っているから、たぶん正しい」

全部自分のコードだと、それが 1 つも来ない

# 見当 1: 直前に手を入れた場所

・64 ビット化で、関数ごとの「変数の置き場」の大きさを書き換えた

・壊れた値の置き場所は、関数が確保した範囲の外だった

・→ 置き場の数を数え間違えて、隣を踏んでいる

# 見当 2: 前に一度壊れた仕組み

・関数を呼ぶ前に値を退避し、戻ったら書き戻す

・この作業の途中で、一度ここが壊れていた

・呼び出しの前後で、値が変わって見えた

・→ 呼ばれた側が、書き戻し損ねている

どちらも、証拠つき

# そして 2 回とも外れた

# 推測を、検査に変える

・コンパイラは関数ごとに「変数を何個ぶん置けるか」を先に決める

・確保した数と、実際に使った数を、コンパイラ自身に比べさせる

・合わなければ、コンパイルを止める

機械が言った

# reserved 2, used 17

確保 2 個、使用 17 個。疑いは当たっていた

直した

溢れは、どのプログラムでもゼロになった

# それでも puts 1 + 1 は通らない

# 実在の欠陥だった。でも、原因ではなかった

推測のままなら「直し方が悪かったのかも」と迷い続けたはず

検査にしてあったから、当たりと原因を切り離せた

## 見当 2 も、検査に

・CPU エミュレータに、関数呼び出しの約束を見張らせる

・呼ぶ前の値を覚えておき、戻ったときに照合する

# 一度も鳴らない

# 検査は、外れも確定させる

外れた分岐が閉じる。だから次に進める

# 観察が、食い違い始めた

・実行中の値を出力するコードを足すほど、結果が互いに矛盾した

・ある監視は「書き換えなし」、別の監視は「書き換えあり」

・推理の結論:「壊れた値は、メモリのどこかにある」

# 推理をやめて、出来上がった機械語を読む

逆アセンブル = 機械語を、人が読める命令の列に戻すこと

壊れていた値を読みにいく、最初の命令は——

# mv a0, s10

メモリからではなく、CPU 自身の置き場（レジスタ s10）から取っている

「メモリのどこか」は、丸ごと誤りだった

# s10 に書く命令は、その関数に 4 つだけ

うち 1 つが、例外から戻るときの書き戻しだった

# 例外の仕組み

・失敗するかもしれない処理の前に、今の値を退避する

・失敗したら、退避した値を書き戻す

・退避には置き場所が要るので、メモリを一区画確保する

よくある作り

# 書いていた先は、区画の外

| 意図 | 実際 | |
|---|---|---|
| 確保した区画 | 15 個ぶん（0〜14） | 15 個ぶん（0〜14） |
| 値の退避先 | 5〜14 | 20〜29 |

区画の外は、まだ誰にも渡していない空き地

# 空き地は、後から払い出される

1. 区画を確保する
2. 値を、区画の外（空き地）に退避する
3. 処理の途中でメモリが要求され、その空き地が払い出される
4. 退避した値の上に、別のデータが書かれる
5. 失敗して書き戻すと、その別のデータが戻ってくる

# 壊れていたのは 1 行

前: 20 + i × 4

先頭から 20 バイト目。区画の中で、正しかった

後: (20 + i) × wsz

先頭から 20 個目。wsz は 1 個のバイト数

32 / 64 ビット両対応のための、機械的な置き換え

答え合わせ

# 2. 自作コンパイラが出した機械語

最初の見当「直前に手を入れた場所」は、方向としては合っていた。同じ作業の、別の 1 行だった

では、なぜ緑だったのか

# テストは、あった

・「例外から戻ると値が戻る」を確かめるテストが、ずっとあった

・退避する値は 8 個。入れ子も、失敗も成功も試していた

・でも、失敗する処理の中でメモリを確保していなかった

確保 0 回なら通る。1 回で壊れる

間違った場所から書き戻すのと

まだ誰も書いていない場所から書き戻すのは

出力が同じ

# 32 ビットも、同じ変更からずっと壊れていた

・症状が出なかっただけ

・64 ビットに移った時点で、32 ビットで回すのをやめていた

・壊れたと気づく手段を、自分で捨てていた

同じ形に、その後も出会った

# 10 月: 印字は、ずっと正しかった

・コンパイラの印字は、出力先の番号を入れ忘れていた

・CPU エミュレータは、その番号を見ずに全部画面に出していた

・出力は正しい。テストは緑

エミュレータを本物に近づけた瞬間、印字が壊れた

# コードにはコメントまであった

「エミュレータは番号を無視する。QEMU は無視しない」

そのコメントの所は直してあった。同じ形の印字と失敗の経路、ほかの 5 箇所は直っていなかった

# 2 つの誤りが重なると、正しい出力になる

・書き戻し先がずれている × まだ誰も上書きしていない

・番号を入れ忘れた × 番号を見ない

比較が見ているのは、出力だけ

# そして、この発表の準備中にも

・9 月、証拠を測り直して「Ruby 層の 162 本一致は誤り」と訂正した

・今月、CI のログを読み直したら、誤りだったのは訂正の方だった

・162 本という数が、別の比較に 2 つあった

記憶ではなく、ログが判定した

# 持ち帰ってほしいこと

自分が正しいと思っていることを、自分以外が判定できる形に置き換える

| やること | なぜ効くか |
| --- | --- |
| 疑いは検査にする | 外れも確定する |
| 黙っている検査を信じすぎない | 見えない場所がある |
| 観察が食い違ったら、成果物を読む | 成果物は構造を述べている |

# 当てるためではなく、外すために効く

いちばん効いたのは、検査が自分の仮説を否定した 2 回だった

# 自分が書いていない層でも

・「下の層は正しい」は、いつでも仮定

・フレームワークでも、データベースでも、クラウドでも

・仮定が外れたときにやることは、変わらない

# ありがとうございました

- Mere: https://merelang.org/
- CPU エミュレータ: https://github.com/284km/memu
- Ruby 処理系: https://github.com/284km/mere-ruby
- この資料: https://284km.github.io/decks/